Luna 2.0 — Train Instructions

This guide covers the two ways Luna gets smarter after setup:

1. **Website knowledge (web RAG).** Luna reads a website and answers from it.
2. **Learning from daily use.** Luna logs conversations, learns which answers and reply styles work, and accepts new facts after you approve them.

Neither step changes the LLM's weights. Luna's model stays the same; what grows is the knowledge, examples and preferences that go into each prompt. That is why training is fast, runs on a Mac or the Pi, and can be undone by deleting a folder.

It assumes Luna is already installed and runs (see `README.md`). Run every command from the project folder.

Part 1 — Website knowledge (web RAG)

What it does

When a website is configured and the machine is online, Luna does five things:

- crawls up to 40 pages of that one site, following links up to two clicks deep
- splits each page into short passages
- turns each passage into a vector with the same model the knowledge base uses
- saves everything to `data/rag_cache/`
- adds the three passages that best match a question to Luna's prompt, next to the matching knowledge-base entries

After the first crawl, the cache answers questions offline. Luna re-crawls only when online and the cache is more than 24 hours old.

Some rules it follows:

- It stays on the configured site and never follows links to other domains.
- It respects the site's `robots.txt`.
- It skips PDFs, images and other files that aren't web pages.
- `data/knowledge.txt` always wins. A question that matches the knowledge base closely is answered from it, and the website is only used as extra context for the LLM.

Step 1: Choose the site

Pick the most useful starting page, for example the department page rather than the college home page. The crawl budget is spent breadth-first from that page.

Step 2: Build the index by hand and check it

This works on the Mac or the Pi, and needs internet.

```
venv/bin/python web_rag.py --url https://tkmce.ac.in/ --refresh
venv/bin/python web_rag.py --url https://tkmce.ac.in/ --status
```

Then ask it the questions visitors ask, and read what comes back:

```
venv/bin/python web_rag.py --url https://tkmce.ac.in/ \
    --query "who is the principal" \
    --query "what departments are there" \
    --query "how do I apply for admission"
```

Each result shows the page it came from, the passage and a similarity score. What to look for:

What you see	What it means	What to do
Right passage, score above ~0.45	good	nothing
Right page, weak score (0.35–0.45)	the page is relevant but wordy	fine; or add the fact to `knowledge.txt`
Nothing returned	the answer isn't in the crawled pages	start the crawl nearer that content, or raise `--max-pages`
Menus, footers, cookie text	the page is mostly layout	point `--url` at a content page

Useful crawl options:

- `--max-pages 80` crawls more pages. Each page adds a few seconds to the crawl and a little memory to the index.
- `--max-depth 3` follows links one click further from the start page.
- `--top-k` and `--min-sim` let you try retrieval settings before putting them in `config.py`.

Step 3: Turn it on in Luna

In `config.py`:

```
WEB_RAG_URL        = "https://tkmce.ac.in/"   # empty string = off
WEB_RAG_MAX_PAGES  = 40      # crawl budget
WEB_RAG_MAX_DEPTH  = 2       # link depth
WEB_RAG_TOP_K      = 3       # passages added to each question
WEB_RAG_MIN_SIM    = 0.35    # ignore passages less similar than this
WEB_RAG_REFRESH_HOURS = 24   # re-crawl when online and older than this
```

Restart Luna. At startup she loads the cache immediately. If she's online and the cache is stale, she re-crawls in the background, so startup never waits for the network.

Step 4: Get the index onto the Pi

`scripts/deploy.sh` does not copy `data/rag_cache/`, so the Pi keeps its own index. Pick one of these:

- **The Pi has internet.** Set `WEB_RAG_URL` and start Luna. She crawls on her own.
- **The Pi is offline.** Build the cache on the Mac (Step 2), then copy it across once:

```
rsync -av data/rag_cache/ luna:/home/luna/Luna/data/rag_cache/
```

When to update what

- Stable, important facts (HOD, lab locations, timings) belong in `data/knowledge.txt`. Those are answered word for word and instantly.
- The website covers everything else: news, events, long descriptions.
- If the website and `knowledge.txt` disagree, fix `knowledge.txt`. It is trusted first.

Part 2 — Learning from daily use

How it works

Every conversation turn is logged, and every turn can earn a **reward** between −1 and +1:

Signal	Reward	How Luna notices
"good answer", "that's right", "perfect"	+1.0	the next thing said
thumbs-up gesture after a reply	+0.6	camera
"thanks", laughter, a compliment	+0.5	the next thing said
they keep talking within 20 s	+0.25	timing
they ask the same question again	−0.5	similarity to the last question
"that's wrong", "no, that's not right"	−1.0	the next thing said
they walk away within 10 s of a reply	−0.2	camera

Rewards update four things, all stored in `data/learning/`:

1. **Reply style.** There are 8 styles, made from three choices: one sentence or two to three; playful or plain and warm; ending with a question or not. Luna keeps a separate score for each situation: known person or stranger, college, general or out-of-scope question, and morning, afternoon or evening. She tries styles evenly at first. After 8 rewarded turns in a situation she leans toward the style that has earned the best rewards, while still trying the others now and then.
2. **Liked answers.** Replies rewarded +0.5 or more are kept, up to 200. The two most similar are shown to the LLM as examples for new questions.
3. **People.** Recognised people get a profile: number of visits, favourite topics and preferred style. When she sees them, a one-line summary goes into the prompt.
4. **Taught facts.** "Remember that…", or the answer given after "that's wrong", goes to a pending list and only becomes knowledge after you approve it (Step 5).

After 10 pm Luna writes a three-sentence **diary** of the day to `data/diary/` and can recall it later ("lots of people asked about Hestia yesterday").

Step 1: Check the logs are being written

Every utterance Luna hears, answered or ignored, is written to `data/logs/conversation_YYYY-MM-DD.jsonl`. Read a day as a transcript:

```
venv/bin/python scripts/review_log.py                # today
venv/bin/python scripts/review_log.py yesterday
venv/bin/python scripts/review_log.py 2026-09-24 --ignored   # include ignored speech
venv/bin/python scripts/review_log.py --rewards      # rewards + style scores
venv/bin/python scripts/review_log.py --json         # raw records
```

Use the transcript to fix problems at the source:

You see	Fix
Wrong or missing college fact	add or correct the line in `data/knowledge.txt` (reloads live)
She answered chatter that wasn't meant for her	raise `ADDR_THRESHOLD`, or set `ADDR_FACE_OR_NAME = True`
She ignored a real question (`--ignored` shows why)	lower `ADDR_THRESHOLD` or `LOUDNESS_NEAR_DB`
She invented something	add the right fact to `knowledge.txt`; if it keeps happening, check `docs/llm_benchmark.md`
Misheard words	better mic placement or gain; `scripts/bench_vosk.py` to test

Logs are kept for 365 days on a Mac and 45 days on the Pi (`CONVO_LOG_RETENTION_DAYS`).

Step 2: Train on the Mac with simulated visitors

Before Luna meets real people, give her a head start. A second local model plays the visitor, with varied roles and goals: a student, parent, faculty member or prospective student, who might be curious, impatient or chatty. A third model judges each reply from 1 to 5. The score becomes a reward (≤2 → −1, 3 → 0, 4 → +0.5, 5 → +1).

```
# a quick check first
venv/bin/python scripts/train_learning.py --simulate 2 --judge-model gemma3:4b

# a real run
venv/bin/python scripts/train_learning.py --simulate 30 --judge-model gemma3:4b
```

Options:

- `--visitor-model` sets the model that plays the visitor (default `llama3.2:3b`).

- `--judge-model` sets the judge. **Use `gemma3:4b` or a larger model.** `llama3.2:3b` was too lenient in testing and approved invented details.
- `--luna-model` sets the model that answers as Luna (default `luna`).
- `--seed` makes a run repeatable.

Simulated sessions are logged to `data/logs_sim/`, never mixed with real logs. Each session takes a minute or two on an M3 Mac, so budget up to an hour for 30 sessions. At the end you get a table of the best style per situation.

Check the result:

```
venv/bin/python scripts/review_log.py --sim --rewards
```

If the judge is rewarding bad answers (invented names, pretending to call someone), throw the run away and retrain with a stronger judge:

```
rm -rf data/learning data/logs_sim
venv/bin/python scripts/train_learning.py --simulate 30 --judge-model gemma3:4b
```

Step 3: Train from real conversations

Once Luna has been used for a while (on the Mac, or with logs copied from the Pi), rebuild her learning state from the real logs:

```
venv/bin/python scripts/train_learning.py --replay                  # real logs only
venv/bin/python scripts/train_learning.py --replay --include-sim    # real + simulated
```

`--replay` wipes and rebuilds the style scores, liked answers and profiles from the logs. Pending facts and the diary are kept.

To copy the Pi's logs to the Mac first:

```
rsync -av luna:/home/luna/Luna/data/logs/ data/logs/
```

Step 4: Send the trained state to the Pi once

```
scripts/deploy.sh --go --seed-learning
```

This copies `data/learning/` to the Pi a single time. After that, normal deploys (`scripts/deploy.sh --go`) never touch the Pi's `data/learning/`, `data/logs/` or `data/diary/`, so everything the Pi learns on its own is kept.

To reset the Pi's learning completely, stop Luna and delete that folder:

```
ssh luna 'rm -rf /home/luna/Luna/data/learning'
```

Step 5: Approve facts people taught her

Nothing people tell Luna becomes "knowledge" until a person approves it. That stops pranks and mistakes from reaching visitors.

```
venv/bin/python scripts/approve_facts.py                  # list pending facts, numbered
venv/bin/python scripts/approve_facts.py --approve 2 "when does the library close"
venv/bin/python scripts/approve_facts.py --reject 3
```

`--approve N "question"` adds `question:fact` to `data/knowledge.txt` and removes it from the pending list.

Important on the Pi. Approving on the Pi changes the Pi's `knowledge.txt`, and the next `deploy.sh --go` from the Mac overwrites it. Always copy approved lines into the Mac's `data/knowledge.txt` too. The script prints a reminder.

A routine that works

When	What
Once, before first use	`train_learning.py --simulate 30 --judge-model gemma3:4b`, then `deploy.sh --go --seed-learning`
Daily (5 min)	`review_log.py yesterday --ignored` on the Pi; fix `knowledge.txt`; `approve_facts.py`
Weekly	copy the Pi logs to the Mac, check `review_log.py --rewards`, adjust `ADDR_*` if needed
When the website changes a lot	`web_rag.py --refresh` (or wait for the daily re-crawl)

Settings

All in `config.py`, section "Learning from daily life":

Setting	Default	Meaning
`LEARNING_ENABLED`	`True`	master switch
`LEARNING_STYLE_ENABLED`	`True`	let learned styles steer replies
`LEARNING_MIN_TURNS_TO_STEER`	8	rewarded turns before a situation's style is trusted
`LEARNING_EXEMPLARS_K`	2	liked answers shown per question
`LEARNING_EXEMPLAR_MIN_REWARD`	0.5	reward needed to keep an answer
`LEARNING_ENGAGEMENT_WINDOW`	20 s	a follow-up within this counts as engaged
`LEARNING_REWARD_*`	see the reward table	the size of each signal
`DIARY_ENABLED`, `DIARY_WRITE_HOUR`	`True`, 22	nightly diary
`CONVO_LOG_RETENTION_DAYS`	365 Mac / 45 Pi	how long logs are kept

Privacy

The logs, profiles and diary record what people said and who said it. They stay on the machine: `.gitignore` keeps `data/logs/`, `data/learning/`, `data/diary/`, `data/faces/` and `data/rag_cache/` out of the repository, and deploys never copy them over. If you run Luna in a public space, put up a short notice that she records conversations, and shorten `CONVO_LOG_RETENTION_DAYS` if needed. "Forget me" removes a person's face. To delete everything Luna remembers about people:

```
rm -rf data/faces data/face_log.jsonl data/logs data/learning data/diary
```

Optional, later: fine-tuning the model

Everything above leaves the LLM untouched. If months of logs show a consistent problem that prompting can't fix, you can fine-tune:

1. Export high-reward turns with `scripts/review_log.py --json`.
2. Review them by hand.
3. Train a LoRA adapter for `llama3.2:3b` on the Mac, for example with MLX or Unsloth.
4. Import the adapter into Ollama with an `ADAPTER` line in the Modelfile.
5. Re-run `scripts/bench_llm.py` and only switch if the score goes up.

This is not needed to get started, and a few hundred noisy turns usually make a model worse, not better.